



Module 6 — Organisation et bonnes pratiques Cypress



Objectif du module

À la fin de ce module, l'étudiant saura :

- organiser un projet Cypress **comme en entreprise**
- éviter les duplications
- écrire des tests lisibles et maintenables
- comprendre **où mettre quoi et pourquoi**
- préparer la suite (tests avancés, projet final, CI/CD)



Phrase clé du module :

Un bon test n'est pas juste un test qui passe, c'est un test qu'on comprend.



Rappel fondamental (à dire à l'étudiant)

Avant ce module, nous avons appris :

- JavaScript
- Cypress de base
- interactions UI
- requêtes API



Maintenant, on apprend à travailler proprement.



Architecture finale du projet (à garder toute la formation)



Document à fournir à l'étudiant



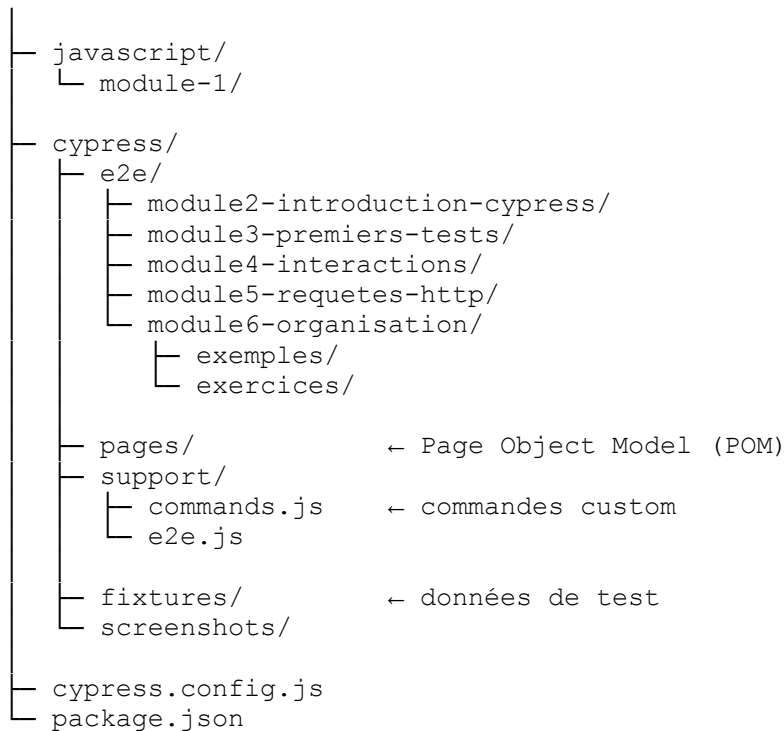
Nom recommandé :

Architecture-du-projet-Cypress.pdf ou .docx



Architecture complète

`cypress-formation/`



Règle d'or rappelée :

- 1 dossier = 1 module
- 1 fichier = 1 idée
- jamais de gros fichiers fourre-tout

2 Page Object Model (POM) — expliqué depuis zéro

C'est quoi le POM ?

Le **Page Object Model** est une manière d'organiser le code en représentant **chaque page de l'application** par une classe **JavaScript**.

Phrase clé (ultra importante) :

Un test ne devrait jamais connaître le HTML.

Pourquoi utiliser le POM ?

Sans POM :

```
cy.get("#email").type("test@mail.com");
cy.get("#password").type("1234");
cy.get("#login").click();
```

- ✗ HTML partout
 - ✗ fragile
 - ✗ illisible
 - ✗ difficile à maintenir
-

Avec POM :

```
LoginPage.saisirEmail("test@mail.com");
LoginPage.saisirMotDePasse("1234");
LoginPage.seConnecter();
```

- ✓ lisible
 - ✓ métier
 - ✓ maintenable
 - ✓ entreprise-ready
-

3 Création d'une page (POM)

 cypress/pages/LoginPage.js

```
class LoginPage {
  visit() {
    cy.visit("https://example.cypress.io/commands/actions");
  }

  saisirEmail(email) {
    cy.get(".action-email").type(email); // Existe bien
  }

  saisirMotDePasse(password) {
    // On utilise le champ 'Disabled' ou 'Focus' pour simuler le
password
    cy.get(".action-focus").type(password);
  }

  cliquerConnexion() {
    // On clique sur le bouton du formulaire d'email
    cy.get('.action-form [type="text"]').first().type('{enter}');
  }
}
export default new LoginPage();
```

4 Explication CRUCIALE : `export default new LoginPage();`

◆ Ce que ça fait exactement

- `class LoginPage` → définit une classe
- `new LoginPage()` → crée une **instance**
- `export default` → permet de l'utiliser ailleurs

👉 On exporte **une instance prête à l'emploi**

◆ Pourquoi ne pas faire juste `export default LoginPage` ?

Si on faisait :

```
export default LoginPage;
```

Alors dans le test :

```
const page = new LoginPage();  
page.visit();
```

- ✗ plus verbeux
- ✗ inutile ici
- ✗ pas pédagogique pour débutant

🧠 **Choix pédagogique volontaire :**

Une page = un objet prêt à être utilisé

5 Utilisation du POM dans un test

 `cypress/e2e/module6-organisation/exemples/login_pom.cy.js`

```
import LoginPage from "../../pages/LoginPage";  
  
describe("Exercice POM", () => {  
  it("devrait afficher un message de succès après l'envoi du formulaire",  
    () => {  
      LoginPage.visit();  
      LoginPage.saisirEmail("test@mail.com");  
      LoginPage.saisirMotDePasse("123456");  
      LoginPage.cliquerConnexion();  
  
      // L'assertion précise basée sur le comportement du site  
      cy.contains("Your form has been submitted!").should("be.visible");  
    });  
});
```

6 Commandes personnalisées (pour éviter les répétitions)

◆ Pourquoi les commandes ?

Certaines actions sont :

- répétées partout
- transverses
- non liées à une seule page

Exemple :

- login
- reset data
- seed utilisateur

◆ Création d'une commande custom

 cypress/support/commands.js

```
Cypress.Commands.add("login", (email, password) => {
  cy.visit("https://example.cypress.io/commands/actions");
  cy.get(".action-email").type(email);
  cy.get(".action-focus").type(password); // Utilisation du champ focus
  cy.get(".action-form").submit(); // Déclenche le message de succès
});
```

◆ Utilisation dans un test

 cypress/e2e/module6-organisation/exemples/login_command.cy.js

```
describe("Test avec login", () => {
  it("utilise la commande login", () => {
    cy.login("admin@test.com", "123456");
    cy.contains("Your form has been submitted!").should("be.visible");
  });
});
```

Phrase clé entreprise :

Une commande Cypress est une action métier réutilisable.

7 POM vs Commandes — quand utiliser quoi ?

Besoin	Utiliser
--------	----------

Actions d'une page POM	
------------------------	--

Besoin	Utiliser
Actions globales	Commandes
Lisibilité métier	Les deux

🧠 Métaphore pédagogique :

- **POM** = télécommande de la page
- **Commande** = raccourci clavier global

8 Hooks (before, beforeEach...) — utilisation raisonnée

❌ Mauvaise pratique

```
beforeEach(() => {
  cy.login();
});
```

- ❌ tous les tests dépendent du login
- ❌ lent
- ❌ fragile

✅ Bonne pratique

```
it("test qui nécessite login", () => {
  cy.login();
  // test
});
```

🧠 Phrase clé à marteler :

Un test doit déclarer explicitement ses dépendances.

9 Gherkin / BDD — introduction (sans implémentation)

◆ C'est quoi ?

Le Gherkin permet d'écrire des scénarios lisibles :

```
Étant donné que je suis connecté
Quand je clique sur Connexion
Alors je vois le tableau de bord
```

- ✓ lisible par les non-tech
- ✓ très utilisé en entreprise

📌 **Mais :**

- pas encore maintenant
- viendra plus tard (module avancé / projet final)

Exercices pratiques

📌 **Site utilisé pour TOUS les exercices**

👉 <https://example.cypress.io/commands/actions>

- ✓ stable
- ✓ déjà utilisé dans les modules précédents
- ✓ aucun login réel requis
- ✓ parfait pédagogiquement

Exercice 1 — Utiliser le Page Object Model (POM)

📁 **Fichier à créer**

`cypress/e2e/module6-organisation/exercices/exercice_pom.cy.js`

👉 **Consigne pour l'étudiant**

1. Utiliser la page `LoginPage`
2. Visiter la page `Actions`
3. Remplir :
 - le champ email
 - le champ mot de passe
4. Cliquer sur le bouton
5. Vérifier que le texte "**Actions**" est visible



- Ne pas utiliser `cy.get()` directement dans le test
- Tout doit passer par le POM

Explication de la correction

- `LoginPage.visit()` → la navigation est centralisée

- Le test **ne connaît pas le HTML**
- Si un sélecteur change → **un seul fichier à modifier**
- Le test est **lisible comme un scénario métier**

📌 **Objectif pédagogique atteint :**

Le test raconte ce que fait l'utilisateur, pas comment la page est construite.

Exercice 2 — Créer et utiliser une commande personnalisée

📁 **Fichier à modifier**

`cypress/support/commands.js`

👉 **Consigne**

Créer une commande Cypress `cy.login()` qui :

1. Visite la page Actions
2. Remplit le champ email
3. Remplit le champ mot de passe
4. Clique sur le bouton

Exercice 3 — Utiliser la commande `cy.login()`

📁 **Fichier à créer**

`cypress/e2e/module6-organisation/exercices/exercice_commande.cy.js`

👉 **Consigne**

1. Utiliser la commande `cy.login`
2. Passer un email et un mot de passe
3. Vérifier que le texte **"Actions"** est visible

🧠 **Explication**

- `cy.login()` encapsule une **action métier**
- Le test devient **très court**
- On peut réutiliser la commande **dans tout le projet**

📌 Cas entreprise :

Quand le login change, on modifie **UNE SEULE FOIS** la commande.

Exercice 4 — Comparer POM vs Commande (réflexion guidée)

👉 Question pour l'étudiant (à l'oral ou écrit)

- Quand utiliser :
 - une **page (POM)** ?
 - une **commande Cypress** ?
-

⚠ Pièges classiques (à bien marteler)

- ❌ Mettre du HTML dans les tests
- ❌ Mettre tout dans `beforeEach()`
- ❌ Créer des commandes pour tout
- ❌ Mélanger POM et logique métier

✅ Préférer :

- POM pour les pages
 - commandes pour les actions transverses
 - tests courts et lisibles
-

✅ Conclusion finale du Module 6

À ce stade, l'étudiant est capable de :

- ✓ structurer un projet Cypress professionnel
- ✓ utiliser le Page Object Model
- ✓ créer des commandes personnalisées
- ✓ écrire des tests maintenables
- ✓ comprendre les choix d'architecture
- ✓ travailler **comme en mission**